My React Learning Journey

Everything I learned while building this React project — what I did, why I did it, and how it actually works. Written in plain language so that future-me (or any other beginner) can read it and actually understand it, not just copy-paste it.

---

12 chapters · 2026-07-08 – 2026-07-16 · compiled 2026-07-18

Contents

CHAPTER 1

Setting up TanStack Router with Authenticated Routes

2026-07-08

What I set out to do

When you create a React app with Vite, you get a single page — one `App.tsx` with a spinning logo and a counter. Real apps aren't like that. Real apps have **pages**: a login page, a dashboard, a settings page, and so on. And some of those pages should only be visible if you're logged in.

The tool that handles "which component shows up for which URL" is called a **router**. I chose **TanStack Router** because it has great TypeScript support and something called *file-based routing*, which I'll explain below.

So in this session I:

1. Deleted the Vite starter code (`App.tsx` , `App.css` , and most of `index.css`)
2. Installed TanStack Router
3. Set up **file-based routing** (routes are created by making files in a folder)
4. Created a **login page** and a **protected (authenticated) area**
5. Learned how to **redirect** users who aren't logged in

Step 1 — Installing the packages

I added three packages (you can see them in `package.json`):

```
"dependencies": {
  "@tanstack/react-router": "...",           // the router itself
  "@tanstack/react-router-devtools": "..."   // a debugging panel (like React DevTools, but for routes)
},
"devDependencies": {
  "@tanstack/router-plugin": "..."           // a Vite plugin — this is what makes file-based routing work
}
```

Why is `router-plugin` a devDependency? Because it's a build tool, not something that runs in the browser. It watches my `src/routes/` folder while I develop and generates code for me (more on that in Step 3). The user's browser never needs it, so it doesn't belong in `dependencies` .

Step 2 — Telling Vite about the router plugin

In `vite.config.ts` I added the plugin:

```
import { tanstackRouter } from '@tanstack/router-plugin/vite'

export default defineConfig({
  plugins: [
    tanstackRouter({
      target: 'react',
      autoCodeSplitting: true,
    }),
    react(),
    // ...
  ],
})
```

Two things worth understanding here:

- **The plugin must come *before*** `react()` in the plugins list. The router plugin transforms my route files first, and then the React plugin processes the result.
- `autoCodeSplitting: true` means each page's code is only downloaded when the user actually visits that page. Without this, a user visiting the login page would also download the code for the dashboard, settings, and every other page. With it, they only get what they need. This is called *code splitting* and it makes apps load faster.

Bonus: the `@` path alias

While I was in the config files, I also set up a **path alias**. Instead of writing imports like this:

```
import { router } from "../../lib/router"   // 😖 how many ../ do I need??
```

I can now write:

```
import { router } from "@/lib"              // 😌 @ always means "the src folder"
```

This needed changes in **two places** (this confused me at first — why two?):

1. `vite.config.ts` — tells *Vite* (the thing that actually bundles and runs my code) what `@` means:

```
resolve: {
  alias: {
    '@': fileURLToPath(new URL('./src', import.meta.url)),
  },
},
```

2. `tsconfig.app.json` — tells *TypeScript* (the thing that checks my types and powers autocomplete in the editor) the same thing:

```
"paths": {
  "@/*": ["./src/*"]
}
```

They're two separate tools, and each one needs to be told independently. If you only do the Vite one, your app runs but your editor shows red squiggly errors. If you only do the TypeScript one, your editor is happy but the app crashes. You need both.

Step 3 — File-based routing: the big idea

This is the coolest part. With TanStack Router, **I don't write a list of routes in code**. Instead, the *files and folders* inside `src/routes/` **are** the routes:

```
src/routes/
├── __root.tsx                → the shell that wraps EVERY page
├── login.tsx                 → the page at /login
├── _authenticated.tsx        → an invisible "gatekeeper" layout (no URL of its own!)
└── _authenticated/
    └── index.tsx             → the page at / (the home page, behind the gatekeeper)
```

When I create a file in this folder, the `router-plugin` I installed in Step 2 notices it and automatically regenerates a file called `src/routeTree.gen.ts`. That file contains the full "map" of all my routes, with full TypeScript types.

> **Important:** `routeTree.gen.ts` is **generated** — I never edit it by hand. The `.gen` in the name is the hint. If I add or delete route files, it updates itself while the dev server is running.

The naming conventions took me a moment to learn:

File name pattern	What it means
`__root.tsx` (two underscores)	The root of everything. Every page renders inside this.
`login.tsx`	A normal route. The file name becomes the URL: `/login`.
`_authenticated.tsx` (one underscore)	A **pathless layout route**. It wraps its children but does **not** add anything to the URL.
`index.tsx`	The "default" page of its folder — like `index.html` on a web server.

The root route — `__root.tsx`

```
import { Outlet, createRootRoute } from '@tanstack/react-router'

export const Route = createRootRoute({
  component: RootComponent,
})

function RootComponent() {
  return (
    <React.Fragment>
      <Outlet />
    </React.Fragment>
  )
}
```

The key concept here is `<Outlet />`. Think of it as a *placeholder* or a *hole* that says: "whatever child route matches the current URL, render it right here." Right now my root component is just the outlet, but later I could add things that should appear on *every* page — a navbar, a footer, a toast container — around it.

Step 4 — The authenticated area (the part I'm most proud of)

I wanted some pages to be **protected**: if you're not logged in, you get kicked to `/login`. Here's the clever part — the file `_authenticated.tsx` starts with a single underscore, which makes it a **pathless layout route**:

- It does **not** appear in the URL. My home page lives at `/`, not at `/_authenticated/`.
- But every route file inside the `_authenticated/` folder renders *inside* it — which means they all have to get past its security check first.

Here's the file:

```tsx
import { createFileRoute, Outlet, redirect } from '@tanstack/react-router'

// temporary — later this will come from real auth state
const isAuthenticated = true;

export const Route = createFileRoute('/_authenticated')({
  beforeLoad: ({ location }) => {
    if (!isAuthenticated) {
      throw redirect({
        to: '/login',
        search: {
          redirect: location.href,
        },
      })
    }
  },
  component: AuthenticatedLayout
})

function AuthenticatedLayout() {
  return <>
    Auth
    <Outlet />
  </>
}
```

Let me break down the pieces that were new to me:

`beforeLoad` — the bouncer at the door

`beforeLoad` runs **before the route's component renders**. It's like a bouncer at a club: you get checked at the door, before you're inside. This is much better than checking auth *inside* the component, because the protected page never even starts rendering for a logged-out user — not even for a split second.

`throw redirect(...)` — wait, you can *throw* a redirect?!

This genuinely surprised me. In JavaScript, `throw` is usually for errors. But TanStack Router uses it as a control-flow trick: throwing immediately stops everything that was about to happen (loading data, rendering the page), and the router catches the thrown redirect and navigates to `/login` instead. It's a clean way to say "STOP. Go here instead."

`search: { redirect: location.href }` — remembering where you were going

This adds a query parameter to the login URL, so it becomes something like:

```
/login?redirect=/dashboard
```

Why? Imagine you click a link to `/dashboard` but you're logged out. You get sent to the login page. After you log in... you'd want to end up on `/dashboard`, the page you originally wanted — not dumped on the home page. By saving the original destination (`location.href`) in the URL, the login page can send you back there after a successful login. I haven't built that "send back" part yet, but the information is already being passed along.

The `isAuthenticated` flag is fake (for now)

```
// temporary
const isAuthenticated = true;
```

Right now it's just hardcoded to `true` so I can build and test the routing structure. Later, this will come from real authentication state (a context, a store, or a token check). Flipping it to `false` is a quick way to test that the redirect works — I tried it, and sure enough, visiting `/` bounced me to `/login`.

The protected home page — `_authenticated/index.tsx`

```
export const Route = createFileRoute('/_authenticated/')({
  component: RouteComponent,
})
```

Because this file lives inside the `_authenticated/` folder, its URL is just `/` (remember: the underscore folder adds no URL segment), but it renders inside `AuthenticatedLayout` and is protected by the `beforeLoad` check. Any new page I drop into that folder gets protection **for free** — I never have to repeat the auth check. That's the whole payoff of this pattern.

Meanwhile `login.tsx` sits *outside* the folder, so it's public — which makes sense, because a logged-out user has to be able to reach the login page!

Step 5 — Wiring the router into the app

Three small files connect everything together.

`src/lib/router.ts` — creating the router

```
import { routeTree } from "@/routeTree.gen"
import { createRouter } from "@tanstack/react-router"

export const router = createRouter({ routeTree })

declare module '@tanstack/react-router' {
  interface Register {
    router: typeof router
  }
}
```

The first part is simple: take the auto-generated route map and create a router from it.

The `declare module` block looked like black magic at first. It's called **module augmentation** — I'm telling TypeScript, globally, "this is MY router, with MY routes." The payoff is amazing type safety everywhere in the app: if I write `<Link to="/logn">` (typo!), TypeScript immediately yells at me because `/logn` is not a route that exists. The router literally knows every valid URL in my app at compile time.

`src/providers/AppProviders.tsx` — a home for all providers

```
import { router } from "@/lib"
import { RouterProvider } from "@tanstack/react-router"

export const AppProviders = () => {
    return (
        <RouterProvider router={router} />
    )
}
```

`RouterProvider` is the component that actually looks at the current URL and renders the matching route. I put it in its own `AppProviders` component instead of directly in `main.tsx`, because React apps tend to accumulate providers over time (theme provider, query client provider, auth provider...). When those arrive, they'll all nest neatly in this one file instead of cluttering the entry point.

`src/main.tsx` — the entry point

```
createRoot(document.getElementById('root')!).render(
  <StrictMode>
    <AppProviders />
  </StrictMode>,
)
```

Notice what's gone: there's no `<App />` anymore, because I deleted `App.tsx` entirely. The router now decides what to render. `main.tsx` stays tiny — it just mounts the app into the page and turns on `StrictMode` (a development helper that warns about common mistakes).

How a request flows through all of this

Putting it all together, here's what happens when someone visits `/`:

1. `main.tsx` mounts `<AppProviders />`, which renders `<RouterProvider />`.
2. The router looks at the URL (`/`) and finds the matching route chain: `__root` → `_authenticated` → `_authenticated/index`.
3. Before rendering anything, `_authenticated`'s `beforeLoad` runs.
   - Logged out? A redirect is thrown, and the user lands on `/login` instead. Done.
   - Logged in? Continue.
4. `__root.tsx` renders, and its `<Outlet />` renders `AuthenticatedLayout`.
5. `AuthenticatedLayout` renders, and *its* `<Outlet />` renders the home page.

It's outlets all the way down — each layer wraps the next.

Things that tripped me up / notes to self

- **Two config files for one alias.** The `@` alias needs to be set in both `vite.config.ts` AND `tsconfig.app.json`. One is for the bundler, one is for the type checker. Forgetting either one causes confusing half-broken behavior.
- **`routeTree.gen.ts` is not mine to edit.** It regenerates automatically. If routes seem "stuck", restarting the dev server regenerates it.
- **Underscore naming matters a lot.** `_authenticated` (one underscore) = pathless layout. `__root` (two underscores) = the special root route. Easy to mistype.
- **`throw redirect()` is intentional**, not an error — that's just how TanStack Router does navigation from inside `beforeLoad`.

What's next

- [ ] Replace the hardcoded `isAuthenticated = true` with real auth state
- [ ] Build an actual login form on `/login`
- [ ] After login, read the `?redirect=` search param and navigate back to it
- [ ] Add the router devtools panel (I installed `@tanstack/react-router-devtools` but haven't rendered it yet)
- [ ] Give `AuthenticatedLayout` a real layout — navbar, sidebar, etc.

CHAPTER 2

Feature Folders & Dynamic Product Routes (Add / Edit / View)

2026-07-08

What I set out to do

Two things in this session:

1. **Organize my code into a `features/` folder** so the project doesn't turn into a mess as it grows.
2. **Build product pages with dynamic URLs** — `/products/add`, `/products/1`, `/products/1/edit` — all powered by **one single component**, `ProductForm`.

Part 1 — The `features/` folder structure

Until now, all my components could just float around anywhere in `src/`. That works for three components, but not for thirty. So I adopted a **feature-based structure**: code is grouped by *what it's for* (home, products, todo), not by *what it is* (components, hooks, utils).

```
src/features/
├── home/
│   ├── components/
│   │   └── Home.tsx
│   └── index.ts          ← the "barrel" (explained below)
├── products/
│   ├── components/
│   │   ├── ProductForm.tsx
│   │   └── ProductList.tsx
│   └── index.ts
└── todo/
    ├── components/
    │   └── TodoList.tsx
    └── index.tsx
```

Why I like this: when I work on "products", *everything* about products is in one folder. Later each feature can grow its own `hooks/`, `api/`, `types/` subfolders without touching the others.

Barrel files (`index.ts`)

Each feature has an `index.ts` that just re-exports its public components:

```
// src/features/products/index.ts
export { ProductList } from './components/ProductList'
export { ProductForm } from './components/ProductForm'
```

This is called a **barrel file**, and it exists purely to make imports nicer. Without it:

```
import { ProductForm } from '@/features/products/components/ProductForm'  // 😖 long and exposes internals
```

With it:

```
import { ProductForm } from '@/features/products'   // 😌 short, and the folder decides what's "public"
```

The barrel acts like the feature's front door. If I later move `ProductForm.tsx` to a different subfolder, only the barrel file changes — every import in the rest of the app keeps working.

Routes stay thin, features hold the real code

An important decision: **route files don't contain UI anymore.** For example, the home page route went from having its own inline component to just pointing at the feature:

```
// src/routes/_authenticated/index.tsx
import { Home } from '@/features/home'
import { createFileRoute } from '@tanstack/react-router'

export const Route = createFileRoute('/_authenticated/')({
  component: Home,
})
```

The `routes/` folder answers "**which URL** shows what?", and the `features/` folder answers "**what does it look like** and how does it behave?". Two different jobs, two different folders.

Part 2 — Dynamic routes with a `$` parameter

Here's the routes folder for products:

```
src/routes/_authenticated/products/
├── index.tsx           →  /products              (product list)
├── add.tsx             →  /products/add          (add a product)
└── $productId/
    ├── index.tsx       →  /products/1            (view product 1)
    └── edit.tsx        →  /products/1/edit       (edit product 1)
```

The new concept is the **`$productId` folder**. A file or folder name starting with `$` is a **dynamic segment**: it's a placeholder that matches *any* value in that position of the URL. Visit `/products/1` and the router captures `productId = "1"`. Visit `/products/abc` and it captures `productId = "abc"`.

"But how does it know `/products/add` isn't a product ID called 'add'?"

This was my first question. The answer: TanStack Router ranks **static segments above dynamic ones**. `add.tsx` is a static route, so `/products/add` matches it first. Only URLs that don't match any static route fall through to `$productId`. I didn't have to configure anything — the naming does it.

Reading the parameter

Inside a route under the `$productId` folder, I get the value with `Route.useParams()`:

```tsx
// src/routes/_authenticated/products/$productId/edit.tsx
import { ProductForm } from '@/features/products'
import { createFileRoute } from '@tanstack/react-router'

export const Route = createFileRoute('/_authenticated/products/$productId/edit')({
  component: ProductEditPage,
})

function ProductEditPage() {
  const { productId } = Route.useParams()
  return <ProductForm mode="edit" productId={productId} />
}
```

The best part: this is fully typed. TypeScript *knows* `productId` exists here, because the `$productId` folder name told the code generator about it. No casting, no "maybe undefined" — the filename **is** the type information.

And because all these files live under the `_authenticated/` folder from entry 01, every product page is automatically behind the login check. I never repeated any auth code.

Part 3 — One `ProductForm`, three pages

I did **not** want three copies of the same form (imagine adding a "description" field and having to do it in three files). So all three pages share one component, and a `mode` prop tells it how to behave:

```
// src/features/products/components/ProductForm.tsx
type ProductFormProps = {
  mode: 'add' | 'edit' | 'view'
  productId?: string          // present for edit + view, absent for add
}

export const ProductForm = ({ mode, productId }: ProductFormProps) => {
  return (
    <>
      <h1>
        {mode == 'edit' ? `Edit mode ${productId}` : mode == 'view' ? `View mode
${productId}` : 'Add mode'}
      </h1>
    </>
  )
}
```

(It only renders a heading for now — the real form fields come later. The point of this session was getting the *structure* right.)

Then each route file is a tiny 5–10 line shell whose only job is translating "which URL am I?" into "which mode + which id?":

Route file	URL	Renders
`add.tsx`	`/products/add`	`<ProductForm mode="add" />`
`$productId/index.tsx`	`/products/1`	`<ProductForm mode="view" productId="1" />`
`$productId/edit.tsx`	`/products/1/edit`	`<ProductForm mode="edit" productId="1" />`

The principle I'm taking away: **routes are thin, components are smart.** The route knows about the URL; the component knows about the behavior. Neither needs to know the other's details.

Two nice TypeScript perks of the `mode` prop being a union type (`'add' | 'edit' | 'view'`):

- My editor autocompletes the three valid values.
- A typo like `mode="edt"` is a compile error, not a silent runtime bug.

Things that tripped me up / notes to self

- `$` **in a filename felt weird** but it's the whole mechanism — the folder name `$productId` is simultaneously the URL pattern AND the TypeScript type for params.
- **Static beats dynamic** in route matching, so `add.tsx` safely coexists with `$productId/`. No special config needed.
- **Barrel files are for the outside world.** Inside the feature I can import directly between files; the barrel is the public doorway for everyone else.

What's next

- [ ] Build the actual form fields in `ProductForm` (inputs, submit handler)
- [ ] Make `mode="view"` render the fields as read-only/disabled
- [ ] Fill in `ProductList` with real products and `<Link>`s to view/edit pages
- [ ] Fetch the product by `productId` in edit/view mode (probably my excuse to learn data fetching properly)

CHAPTER 3

Todo List: useState, Events, and the "Never Mutate" Rule

2026-07-08

What I set out to do

Build the classic beginner exercise — a todo list with **add, edit, and delete** — at `/todo`. It sounds trivial, but it turned out to be the best React lesson so far, because I hit (and fixed) two very common beginner bugs along the way. This entry documents the final code in `src/features/todo/components/TodoList.tsx` *and* the mistakes I made getting there, because the mistakes taught me more than the working code did.

The state: three `useState` hooks

```
type TodoItem = {
    id: number,
    name: string
}

const [todos, setTodos] = useState<TodoItem[]>([]);
const [todoInput, setTodoInput] = useState<string>('');
const [currentTodoId, setCurrentTodoId] = useState<number | null>();
```

Each piece of state has one clear job:

- **`todos`** — the list itself, an array of `TodoItem` objects.
- **`todoInput`** — whatever is currently typed in the text box.
- **`currentTodoId`** — this one is clever: it doubles as the *mode switch*. When it's `null`, the Add button adds a new todo. When it holds an id, the same button *updates* that todo instead. One input + one button = both "add" and "edit", depending on this value.

The controlled input

```
<input type="text" value={todoInput} onChange={(e) => setTodoInput(e.target.value)} />
```

This pattern is called a **controlled input**: the input doesn't own its text — React state does. `value={todoInput}` pushes state *into* the box, and `onChange` pushes every keystroke *back* into state. The payoff shows up in `editTodo`: I can pre-fill the box just by calling `setTodoInput(todo.name)`. If the input owned its own text, I couldn't do that.

Seeding initial data with `useEffect`

```
useEffect(() => {
    setTodos([{id: 1, name: 'First'}])
}, [])
```

`useEffect` with an **empty dependency array** (`[]`) runs once, after the component first appears on screen. Right now I use it to fake some initial data; later this is exactly where fetching from an API would go. (Note: in dev, `StrictMode` deliberately runs effects twice to expose bugs — that's normal, not broken.)

Add + edit in one function

```
const addTodoItem = () => {
    if (!todoInput.trim()) return;

    if (currentTodoId) {
        setTodos(todos.map(todo =>
            todo.id === currentTodoId ? { ...todo, name: todoInput } : todo
        ));
        setCurrentTodoId(null);
    }
    else {
        setTodos([...todos, {
            id: todos.length + 1,
            name: todoInput
        }])
    }
    setTodoInput('');
}
```

Walking through it:

- **The guard clause** `if (!todoInput.trim()) return;` — bail out early on empty or whitespace-only input. Guard clauses at the top keep the rest of the function clean.
- **Edit branch**: `.map()` builds a **new** array where the matching todo is replaced by a **new** object (`{ ...todo, name: todoInput }` = "copy all fields, override `name` ") and every other todo passes through unchanged.
- **Add branch**: `[...todos, newItem]` builds a new array with the old items plus one.
- Either way, finish by clearing the input.

🐛 Bug #1 I hit: mutating state directly

My first version of the edit branch was this:

```
// ❌ WRONG — my first attempt
const todoItem = todos.find(todo => todo.id === currentTodoId);
todoItem.name = todoInput;
```

Looks reasonable, right? Find the item, change its name. But `find()` returns a **reference** to the actual object inside React's state — so this line secretly edits state **without calling** `setTodos`. And React only re-renders when a setter hands it a **new** array; it never inspects the contents of the old one. So my change was invisible to React. (It *appeared* to work only because `setTodoInput('')` on the next line happened to trigger a render — a coincidence, not correctness.)

The rule I learned: never assign into state. Always build a new array/object and pass it to the setter. Each operation has a standard tool:

Operation	Pattern
Add	`setTodos([...todos, newItem])`
Edit one	`setTodos(todos.map(t => t.id === id ? { ...t, name: newName } : t))`
Delete one	`setTodos(todos.filter(t => t.id !== id))`

No `push`, no `splice`, no `item.field = x`. New arrays, every time.

Edit and delete

```
const editTodo = (todo: TodoItem) => {
    setCurrentTodoId(todo.id);
    setTodoInput(todo.name);
}

const deleteTodo = (id: number) => {
    setTodos(todos.filter(todo => todo.id !== id))
}
```

- `editTodo` doesn't change the list at all! It just flips the component into "edit mode": remember which id we're editing, and pre-fill the input. The actual saving happens later in `addTodoItem`.
- `deleteTodo` uses `.filter()`, which builds a new array containing every todo *except* the matching one — the immutable way to delete.

Rendering the list

```
{todos.map((todo) => (
    <div key={todo.id}>
        {todo.name}{' '}
        <button onClick={() => editTodo(todo)}>Edit</button>{' '}
        <button onClick={() => deleteTodo(todo.id)}>Delete</button>
    </div>
))}

{todos.length < 1 && <h1>No Items</h1>}
```

Three concepts packed in here:

`key={todo.id}`

When rendering a list, React needs a stable identity for each item so it can tell "item moved" apart from "item changed". The `key` prop is that identity. It should be the item's **id**, not the array index — indexes shift when items are deleted, which confuses React into recycling the wrong DOM.

🐛 Bug #2 I hit: calling the handler instead of passing it

My first version was:

```
// ❌ WRONG — my first attempt
<button onClick={editTodo(todo)}>Edit</button>
```

TypeScript stopped me with: *"Type 'void' is not assignable to type 'MouseEventHandler'"*. The problem: `onClick` wants **a function to call later, when the click happens**. But `editTodo(todo)` — with parentheses — runs *immediately, during render*, and hands `onClick` the function's return value (`void` , i.e. nothing). Worse, running `setCurrentTodoId` during render would cause an infinite re-render loop.

The fix is to wrap it in an arrow function, which *is* a function React can call later:

```
// ✅ RIGHT
<button onClick={() => editTodo(todo)}>Edit</button>
```

My rule of thumb now:

- No arguments needed → pass the function itself: `onClick={addTodoItem}`
- Arguments needed → wrap it: `onClick={() => editTodo(todo)}`

If you ever see parentheses directly inside `onClick={...}` , it's almost certainly running too early.

Conditional rendering with `&&`

```
{todos.length < 1 && <h1>No Items</h1>}
```

In JSX, `condition && <Something />` renders the element only when the condition is true. It works because `&&` short-circuits: if the left side is false, the right side never evaluates, and React renders `false` as nothing.

Wiring it to a route

Same "thin route" pattern as the products pages — the route file is just a pointer:

```
// src/routes/_authenticated/todo/index.tsx
import { TodoList } from '@/features/todo'
import { createFileRoute } from '@tanstack/react-router'

export const Route = createFileRoute('/_authenticated/todo/')({
  component: TodoList,
})
```

Living under `_authenticated/` means the todo page is automatically login-protected.

Things that tripped me up / notes to self

- **Mutation is the silent killer.** `todoItem.name = x` compiles fine, sometimes even *looks* like it works, and is still wrong. React only reacts to new references passed through setters. `map` / `filter` / spread are the everyday tools.
- **`onClick={fn(arg)}` runs NOW; `onClick={() => fn(arg)}` runs on click.** The TypeScript error about `void` and `MouseEventHandler` means exactly this.
- **`id: todos.length + 1` is a known weak spot.** With 3 todos (ids 1,2,3), deleting one makes length 2, so the next todo gets id 3 — a duplicate, which breaks both editing and React keys. A never-decreasing counter or `crypto.randomUUID()` fixes it. Leaving this on the to-fix list.
- **Keys should be ids, not array indexes.**

What's next

- [ ] Fix the duplicate-id bug (switch to `crypto.randomUUID()` and a string id)
- [ ] Submit on Enter key, not just the button
- [ ] Change the button label to "Update" while `currentTodoId` is set, plus a "Cancel edit" button
- [ ] Persist todos (localStorage first, then a real API)

CHAPTER 4

The API Layer: Env Config, Axios, TanStack Query & a Mock Server

2026-07-08

What I set out to do

So far my app has been all frontend — routes and components with hardcoded data. Real apps talk to a **backend API**. In this session I built everything the app needs to do that properly:

1. **Environment variables** — so the API URL isn't hardcoded, validated with Zod
2. **An axios API client** — one configured HTTP client for the whole app, with interceptors that attach the auth token and handle "you're logged out" responses
3. **TanStack Query** — the library that will manage server data (caching, loading states, refetching)
4. **A mock Express server** — a fake backend with login, users, and products, so I can build the frontend without waiting for a real backend to exist

The pieces stack like this:

```
Component  →  TanStack Query  →  apiClient (axios)  →  mock server (Express)
                (caching)          (auth header,         (fake database)
                                    base URL)
```

Part 1 — Environment variables (`.env` + Zod validation)

Why not just hardcode the URL?

The API lives at `http://localhost:3000` on my machine, but in production it'll be a real domain. Values that change per environment belong in **environment variables**, not in code.

With Vite, env variables live in `.env` files, and only variables prefixed with `VITE_` are exposed to frontend code (that prefix is a safety feature — it stops you accidentally leaking server secrets into the browser bundle):

```
# .env.example
VITE_APP_ENV=dev
VITE_API_BASE_URL=http://localhost:3000
VITE_API_TIMEOUT=30000
```

Note the split between two files:

- `.env.example` — committed to git. Contains no secrets, just documents *which* variables the project needs. A new developer copies it to get started.
- `.env.local` — my actual values, **git-ignored** (I added it to `.gitignore`). Never committed.

Validating env vars with Zod (`src/config/env.ts`)

Env variables are just strings, and they might be missing or malformed. Instead of finding that out with a weird crash deep inside axios, I validate them **once, at startup**, using Zod:

```
import { z } from 'zod';

const envSchema = z.object({
  VITE_API_BASE_URL: z.url('API base URL must be a valid URL'),
  VITE_API_TIMEOUT: z.string().transform((val) => Number.parseInt(val, 10)),
  VITE_APP_ENV: z.enum(['dev', 'prod', 'qa']),
});
```

Cool details I learned:

- `.transform()` — env vars are always strings, but I want `timeout` as a number. The schema converts it during validation, so the rest of the app never sees the string.
- `z.enum([...])` — `VITE_APP_ENV` can *only* be `dev`, `prod`, or `qa`. A typo like `developement` fails loudly at startup instead of silently misbehaving.
- **Fail fast** — if anything is invalid, the app throws immediately with a readable list of what's wrong, instead of limping along with `undefined` values.

The validated result is exported as a typed `config` object, so everywhere else I write `config.api.baseUrl` with full autocomplete — no raw `import.meta.env` access scattered around the app.

Part 2 — The axios API client (`src/lib/apiClient.ts`)

Instead of calling `axios.get(...)` with the full URL everywhere, I create **one configured instance** that every request goes through:

```
export const apiClient = axios.create({
  withCredentials: true,
  baseURL: config.api.baseUrl,
  timeout: config.api.timeout,
  headers: {
    'Content-Type': 'application/json',
  },
});
```

Now `apiClient.get('/products')` automatically means `GET http://localhost:3000/products` with a 30s timeout. One place to configure, one place to change.

Interceptors — the powerful part

Interceptors are functions that run on **every** request or response passing through the client. Like middleware for HTTP.

Request interceptor — attach the auth token to every outgoing request:

```
apiClient.interceptors.request.use((requestConfig) => {
  const token = "dummy";   // temporary — will come from real auth state later
  if (token) {
    requestConfig.headers.Authorization = `Bearer ${token}`;
  }
  return requestConfig;
});
```

Without this, every single API call in the app would have to remember to add the header itself. With it, authentication is invisible to the rest of the code.

Response interceptor — handle "you're not logged in" globally:

```
apiClient.interceptors.response.use(
  (response) => response,
  (error: AxiosError) => {
    if (error.response?.status === 403) {
      window.location.href = '/login';
    }
    return Promise.reject(error);
  },
);
```

If *any* request comes back with **403 Unauthorized** (expired/invalid session), the user gets sent to the login page — no matter which component made the call. This pairs with the `beforeLoad` guard from entry 01: the router guard protects *navigation*, the interceptor protects *data calls*.

Part 3 — TanStack Query setup

TanStack Query (React Query) manages **server state**: it caches responses, tracks loading/error states, and refetches when data goes stale. I haven't written my first `useQuery` yet — this session was just the setup.

The query client (`src/lib/queryClient.ts`)

```
export const queryClient = new QueryClient({
  defaultOptions: {
    queries: {
      staleTime: 60 * 1000,
      gcTime: 5 * 60 * 1000,
      retry: 1,
      refetchOnWindowFocus: false,
    },
  },
});
```

What each default means (in human words):

- `staleTime: 60 * 1000` — after fetching, data is considered "fresh" for 1 minute. Fresh data is served straight from cache with **no network request**. Navigate away from the products page and back within a minute → instant, no spinner.
- `gcTime: 5 * 60 * 1000` — unused cached data is kept in memory for 5 minutes before being garbage-collected. Between 1 and 5 minutes you get the *stale* data shown instantly while a refetch happens in the background.
- `retry: 1` — a failed request is retried once before showing an error (the default of 3 makes failures feel slow during development).
- `refetchOnWindowFocus: false` — by default, TanStack Query refetches everything whenever you click back into the browser tab. Great for dashboards, noisy while developing, so I turned it off.

Wiring it into `AppProviders`

This is exactly why I made `AppProviders` its own component back in entry 01 — "providers accumulate over time." The second one has arrived:

```
export const AppProviders = () => {
    return (
        <QueryClientProvider client={queryClient}>
            <RouterProvider router={router} />
        </QueryClientProvider>
    )
}
```

`QueryClientProvider` wraps the router so that **every routed page** can call `useQuery` / `useMutation`. `main.tsx` didn't change at all.

I also added `@tanstack/eslint-plugin-query` (dev dependency) — an ESLint plugin that catches common TanStack Query mistakes, like a broken query key.

Part 4 — The mock Express server (`mock-server/server.js`)

I don't have a real backend, and I don't want to build the frontend against thin air. So: a small **Express** server that pretends to be my backend. It's a plain JavaScript file, started separately from the frontend:

```
npm run mock-api    # terminal 1 → API on http://localhost:3000
npm run dev         # terminal 2 → app on http://localhost:5173
```

Express and cors are **devDependencies** — they're development tooling, never shipped to users.

The "database" is just arrays

```
let users = [
  { id: 1, name: 'Sandip', email: 'sandip@example.com', password: 'password123' },
]
let products = [
  { id: 1, name: 'Keyboard', price: 2500, description: 'Mechanical keyboard' },
  { id: 2, name: 'Mouse', price: 1200, description: 'Wireless mouse' },
]
```

In-memory data: restart the server, everything resets to the seed data. For a mock, that's a feature — a clean slate whenever I need one. There's a ready-made login: `sandip@example.com` / `password123` .

The endpoints

Method & path	What it does	Status codes
`POST /auth/login`	Returns `{ token, user }` — the dummy token	200, 400, 401
`POST /auth/register`	Creates a user (public)	201, 400, 409
`GET /users`, `GET /users/:id`	List / view users	200, 404
`POST /users`	Add a user	201, 400, 409
`PUT /users/:id`	Edit a user	200, 404, 409
`DELETE /users/:id`	Delete a user	204, 404
`GET /products`, `GET /products/:id`	List / view products	200, 404
`POST /products`	Add a product	201, 400
`PUT /products/:id`	Edit a product	200, 404
`DELETE /products/:id`	Delete a product	204, 404

Even though it's fake, it uses **real HTTP semantics**: 400 for missing fields, 404 for unknown ids, 409 Conflict for duplicate emails, 204 No Content after a delete. Building the frontend against realistic responses now means fewer surprises when a real backend arrives.

Concepts the server taught me

CORS. The app (port 5173) and the API (port 3000) are different *origins*, and browsers block cross-origin requests by default. The server must explicitly allow it:

```
app.use(cors({
  origin: 'http://localhost:5173',
  credentials: true,
}))
```

`credentials: true` is required because my `apiClient` sets `withCredentials: true` — with credentials involved, the browser refuses the wildcard `origin: '*'`; the origin must be spelled out exactly.

Middleware. `requireAuth` runs *before* the route handler on protected routes — same "bouncer at the door" idea as the router's `beforeLoad`, but on the server side:

```
const requireAuth = (req, res, next) => {
  const header = req.headers.authorization
  if (!header || !header.startsWith('Bearer ')) {
    return res.status(403).json({ message: 'Unauthorized: missing or invalid token' })
  }
  next()  // "you may pass" — continue to the actual route handler
}

app.get('/products', requireAuth, (req, res) => { ... })
```

Any `Bearer <whatever>` is accepted — it's a mock, the point is exercising the frontend's header-sending and error-handling paths, not real security.

Never send passwords back. Even in a mock, user responses strip the password first. This destructure-and-drop trick is neat:

```
const toPublicUser = ({ password, ...user }) => user
```

It pulls `password` out and collects *everything else* into `user` — so the returned object is the user minus the password. Good habit even when the data is fake.

Update without mutation. Editing a product builds a new object and a new array — the same immutability idea as React state (entry 03), which apparently is just a good way to handle data everywhere:

```
const updated = {
  ...product,
  ...(name !== undefined && { name }),   // only override fields that were sent
  ...(price !== undefined && { price }),
}
products = products.map((p) => (p.id === id ? updated : p))
```

That `...(condition && { field })` spread trick means a `PUT` with only `{ price }` updates the price and leaves the other fields alone — a partial update.

Things that tripped me up / notes to self

- **The `VITE_` prefix is mandatory.** An env var without it simply doesn't exist in frontend code. It's a guard against leaking secrets into the bundle.
- **`.env.example` is committed, `.env.local` is git-ignored.** Template vs real values.
- ⚠️ **401 vs 403 mismatch to resolve:** my `requireAuth` middleware currently returns **403** for a missing/invalid token, but the axios interceptor only redirects to `/login` on **401**. HTTP

convention: 401 = "not authenticated (who are you?)", 403 = "authenticated but not allowed (I know who you are — no)". So for a missing token, 401 is the conventional choice — and it's what my interceptor listens for. Either the middleware should return 401, or the interceptor should also handle 403. *(Update: resolved in entry 12 — I ended up standardizing on **403** on both sides, not 401.)*

- **zod is missing from `package.json`.** `env.ts` imports it, and it works only because some other package happened to install it transitively. On a fresh clone it would break. Fix: `npm install zod`.
- **The token is still fake everywhere** — the interceptor hardcodes `"dummy"`, and login isn't wired to a form yet. Real auth state is the next big topic.

What's next

- [ ] `npm install zod` to make it an explicit dependency
- [x] Fix the 401/403 mismatch between mock server and interceptor — settled on 403 (entry 12)
- [ ] Build the login form and call `POST /auth/login`, store the returned token
- [ ] Replace the hardcoded `"dummy"` token in the request interceptor with the real one
- [ ] First `useQuery`: fetch `/products` in `ProductList`
- [ ] First `useMutation`: wire `ProductForm` to POST/PUT/DELETE

CHAPTER 5

The Products Data Layer: My First useQuery & useMutation

2026-07-09

What I set out to do

Entry 04 built all the plumbing — the axios client, the query client, the mock server — but nothing actually *used* it yet. This session I built the **data layer for the products feature**: typed functions that call the API, and TanStack Query hooks that components will use to read and change products.

Three new files, one job each:

```
src/features/products/
├── product.types.ts     → what a product IS        (the shape)
├── products.api.ts      → how to fetch/change one  (raw HTTP calls)
└── product.query.ts     → how components use it    (useQuery/useMutation hooks)
```

This layering continues the theme from entry 02 ("routes are thin, components are smart"): now it's **components are thin, hooks are smart**. A component will just say `useProducts()` and get data + loading + error states — it won't know axios exists.

Part 1 — The type (`product.types.ts`)

```ts
export interface IProduct {
  id: number;
  name: string;
  price: number;
  description: string;
}
```

One interface, describing what the mock server returns. Every layer above imports this, so if a product ever gains a field, I add it in exactly one place and TypeScript points out everywhere that needs updating.

Part 2 — The API functions (`products.api.ts`)

Each function is a thin, typed wrapper around one endpoint:

```ts
import { apiClient } from "@/lib";
import type { IProduct } from "./product.types";

export const getProducts = async () => {
  const res = await apiClient.get<IProduct[]>('/products');
  return res.data;
};

export const getProduct = async (id: number) => {
  const res = await apiClient.get<IProduct>(`/products/${id}`);
  return res.data;
};

export const createProduct = async (body: IProduct) => {
  const res = await apiClient.post<IProduct>('/products', body);
  return res.data;
};

export const updateProduct = async (id: number, body: IProduct) => {
  const res = await apiClient.put<IProduct>(`/products/${id}`, body);
  return res.data;
};

export const deleteProduct = async (id: number) => {
  await apiClient.delete(`/products/${id}`);
};
```

Things worth noticing:

- **The generic tells TypeScript what comes back.** `apiClient.get<IProduct[]>(...)` means "the response body is an array of products" — so `res.data` is fully typed and autocomplete works downstream. (It's a *promise*, not a *check* — TypeScript trusts me here; nothing verifies the server actually sent that shape. Runtime validation with Zod is a possible future upgrade.)
- **Return `res.data`, not `res`.** Axios wraps responses in an object carrying status, headers, etc. The rest of the app only cares about the body, so the API layer unwraps it once, here.
- **All the entry-04 plumbing kicks in silently.** Because these go through `apiClient`, every call automatically gets the base URL, the timeout, the Bearer token header, and the 403 → `/login` redirect. These functions stay tiny because the interceptors do the boring work.
- I also had to add `apiClient` to the `src/lib` barrel (`src/lib/index.ts`) so it's importable as `@/lib` — the barrel only exposes what's been explicitly exported.

Part 3 — The query hooks (`product.query.ts`)

This is the new mental model, so I'll go slowly.

Query keys — the cache's addresses

```
export const productKeys = {
    all: ['products'] as const,
};
```

TanStack Query caches every query under a **key** (an array). The key is like a label on a drawer: `['products']` is the drawer where the product list lives. Ask for the same key twice → same drawer, no second network request while fresh.

Defining keys in one `productKeys` object (instead of typing `['products']` inline everywhere) matters because *invalidation* — telling the cache "this drawer is out of date" — must use the **exact same key** as the query. A typo like `['product']` in one place would silently break the refresh behavior. One shared constant makes that impossible. (`as const` keeps the type exact, which TanStack Query likes.)

Reading data: `useQuery`

```
export const useProducts = () ⇒
    useQuery({
        queryKey: productKeys.all,
        queryFn: getProducts,
    });
```

`useQuery` takes "where to store it" (`queryKey`) and "how to fetch it" (`queryFn`). In exchange, a component gets a whole bundle:

```
const { data, isPending, isError, error } = useProducts();
```

...and TanStack Query handles what I'd otherwise hand-roll with `useEffect` + three `useState` s: fetch on mount, track loading, track errors, cache the result, dedupe simultaneous requests, refetch when stale (per the 1-minute `staleTime` from entry 04). Declarative data: *"I need products"*, not *"go fetch products now."*

Wrapping it in a custom hook (`useProducts`) instead of calling `useQuery` directly in components keeps the key + fetcher pairing in exactly one place.

Changing data: `useMutation`

Queries are for *reading*; **mutations** are for *writing* (create/update/delete). They don't run on mount — they run when you call them:

```
export const useCreateProduct = () => {
    const queryClient = useQueryClient();
    return useMutation({
        mutationFn: (body: IProduct) => createProduct(body),
        onSuccess: () => {
            queryClient.invalidateQueries({ queryKey: productKeys.all });
        },
    });
};
```

In a component this will look like:

```
const createMutation = useCreateProduct();
createMutation.mutate(newProduct);       // fire it, e.g. in the form's onSubmit
// createMutation.isPending → disable the submit button while saving
```

`invalidateQueries` — the part that makes it all click

Here's the problem it solves. Suppose the product list is cached, and I add a new product. The server now has 3 products; my cache still says 2. Stale UI.

The fix is that `onSuccess` line: after the mutation succeeds, `invalidateQueries({ queryKey: productKeys.all })` tells the cache *"whatever you have under `['products']` is now out of date."* TanStack Query then automatically refetches it (immediately, if any component on screen is using it), and every component showing the list re-renders with fresh data.

That's the whole loop:

```
mutate() → server changes → onSuccess → invalidate ['products'] → auto refetch → UI updates
```

No manual `setProducts([...products, newOne])` state juggling, no passing callbacks between pages. The form page and the list page don't even need to know about each other — they're connected only through the query key.

`useUpdateProduct` and `useDeleteProduct` follow the identical pattern. One quirk: `mutationFn` accepts only **one argument**, so the update hook bundles its two values into a single object:

```
mutationFn: ({ id, body }: { id: number; body: IProduct }) => updateProduct(id, body),
// called as: updateMutation.mutate({ id: 5, body: editedProduct })
```

Things that tripped me up / notes to self

- **The query key is the contract.** Query and invalidation must use the same key — that's why `productKeys` exists. As the app grows, this becomes a "key factory" with entries like `productKeys.detail(id)` for single products.
- ⚠️ **`createProduct` takes a full `IProduct` — including `id`.** But the server generates the id itself and ignores whatever I send. The honest type would be `Omit<IProduct, 'id'>` ("an IProduct minus its id"). Works fine today, but the type is lying slightly. On the fix list.
- ⚠️ **`getProduct` has no hook yet.** The API function exists, but there's no `useProduct(id)` — which the view/edit pages need. It'll want its own cache address per product, something like `['products', id]`.
- **TypeScript generics on axios are trust, not validation.** `get<IProduct[]>` types the response but never checks it at runtime. Zod could close that gap later.
- **Minor naming inconsistency:** `products.api.ts` (plural) vs `product.query.ts` / `product.types.ts` (singular). Harmless, but worth picking one style before more features copy the pattern.

What's next

- [ ] Wire `useProducts()` into `ProductList` — render data, `isPending`, `isError`
- [ ] Add `useProduct(id)` with key `['products', id]` for the view/edit pages
- [ ] Wire `ProductForm` to `useCreateProduct` / `useUpdateProduct` (and use `isPending` to disable the submit button)
- [ ] Change `createProduct`'s body type to `Omit<IProduct, 'id'>`
- [ ] Delete button on the list using `useDeleteProduct`

CHAPTER 6

Giving the Mock Server a Real(ish) Database with db.json

2026-07-10

What I set out to do

Back in entry 04 my mock server kept everything in **in-memory arrays** — plain `let users = [...]` and `let products = [...]` variables. That worked, but it had one annoying property: every time I restarted the server, all my changes vanished and it reset to the two seed products. Add a product, restart, gone.

This session I fixed that by moving the data into a `db.json` **file** that the server reads from and writes back to. Now data survives restarts. I also filled it with **50 products** so the product list has something realistic to show.

Part 1 — The data file (`mock-server/db.json`)

The whole "database" is now just a JSON file:

```
{
  "users": [
    { "id": 1, "name": "Sandip", "email": "sandip@example.com", "password": "password123" }
  ],
  "products": [
    { "id": 1, "name": "Keyboard", "price": 2500, "description": "Mechanical keyboard" },
    { "id": 2, "name": "Mouse", "price": 1200, "description": "Wireless mouse" }
    // ... 48 more, up to id 50
  ]
}
```

It has the same shape as my old in-memory data — a `users` array and a `products` array — just living on disk instead of in a variable.

Part 2 — Reading and writing the file

At the top of `server.js` I added two tiny helper functions:

```
import { readFileSync, writeFileSync } from 'fs'
import { fileURLToPath } from 'url'
import { dirname, join } from 'path'

const __dirname = dirname(fileURLToPath(import.meta.url))
const DB_PATH = join(__dirname, 'db.json')

const readDb = () => JSON.parse(readFileSync(DB_PATH, 'utf-8'))
const writeDb = (db) => writeFileSync(DB_PATH, JSON.stringify(db, null, 2))
```

Let me unpack the parts that were new to me:

Recreating `__dirname` in an ES module

I wanted to load `db.json` from the same folder as `server.js`. Normally you'd use `__dirname` (Node's "the folder this file lives in"), but `__dirname` **doesn't exist in ES modules** — and my project uses `"type": "module"` (from entry 01), so all my files are ES modules. So I had to rebuild it:

- `import.meta.url` — the URL of the current file (something like `file:///C:/.../server.js`)
- `fileURLToPath(...)` — turns that URL into a normal file path
- `dirname(...)` — chops off the filename, leaving just the folder

Then `join(__dirname, 'db.json')` builds the full path to the data file. Using `join` instead of gluing strings with `/` means it works on Windows and Mac/Linux alike.

`readDb` / `writeDb`

- **`readFileSync(DB_PATH, 'utf-8')`** reads the file as text, and `JSON.parse` turns that text into real JavaScript objects/arrays.
- **`writeDb`** does the reverse: `JSON.stringify(db, null, 2)` turns the data back into text (the `2` means "pretty-print with 2-space indents" so the file stays human-readable), and `writeFileSync` saves it.

I'm using the **`Sync`** versions on purpose. Normally in Node you avoid synchronous file calls because they block everything, but for a tiny local mock server it keeps the code simple and readable — no `async` / `await` or callbacks. Not something I'd do in a real production backend.

Part 3 — The pattern in every route

The rule is simple and consistent:

- **Reading data (GET)** → `readDb()` at the start, then respond.
- **Changing data (POST / PUT / DELETE)** → `readDb()`, modify, then `writeDb()` to save it back.

For example, adding a product went from touching an in-memory array to this:

```
app.post('/products', requireAuth, (req, res) => {
  const { name, price, description } = req.body ?? {}
  if (!name || price === undefined) {
    return res.status(400).json({ message: 'name and price are required' })
  }

  const db = readDb()                                    // load current data
  const product = { id: nextId(db.products), name, price, description: description ?? '' }
  db.products.push(product)                              // change it
  writeDb(db)                                            // save back to disk
  res.status(201).json(product)
})
```

The read-modify-write shape is the same for users too. Every write ends with `writeDb(db)` — forget that line and the change would only live in memory and vanish on restart (which is exactly the old behavior I was trying to escape).

Generating ids from the file

The old code used counter variables (`let nextProductId = 3`). But counters reset to their starting value every restart — which would cause **duplicate ids** once the file has more data than the counter knows about. So I compute the next id from the data itself:

```
const nextId = (items) => items.reduce((max, item) => Math.max(max, item.id), 0) + 1
```

In plain words: "look at every item, find the biggest `id` , add 1." Start the running maximum at `0` so an empty list correctly gives id `1` . Because this reads the actual file every time, it stays correct across restarts — no counter to fall out of sync.

This is the same "derive it, don't store it separately" lesson as React keys in entry 03: a value you compute from the source of truth can't drift out of sync with it.

Things that tripped me up / notes to self

- ⚠️ **`db.json` is now a *live* file that the app rewrites.** This is a double-edged sword. It's great that data persists — but it also means my seed data gets overwritten as I click around. If I ever want the original 50 products back, I'd need a separate seed file (e.g. `db.seed.json`) that I copy over `db.json` to reset. Right now there's no reset mechanism. Also worth thinking about whether `db.json` should even be committed to git, since it'll be full of my test junk.
- 🐛 **A zombie server wasted 10 minutes.** While testing, I saw completely wrong data come back — a product I never added, and a missing one. The cause: an **old server process from a previous run was still holding port 3000**, so my requests were hitting the stale in-memory

server, not my new file-backed one. Lesson: if the data looks impossible, check for a leftover process:

```
netstat -ano | findstr :3000      # find the PID listening on 3000
taskkill /F /PID <pid>            # kill it
```

Only one process can own a port, so a "ghost" server silently answering requests is a real gotcha.

- **This supersedes entry 04's description of the server.** Entry 04 documented the server as in-memory; from now on it's file-backed. The endpoints, status codes, and `requireAuth` behavior are all unchanged — only *where the data lives* changed.

What's next

- [ ] Add a `db.seed.json` + a `reset` script so I can restore the 50 products
- [ ] Decide whether to git-ignore `db.json`
- [ ] (Maybe) switch to async `fs.promises` reads/writes as a learning exercise

CHAPTER 7

Wiring the Products UI to the Data Layer (List, View, Delete)

2026-07-10

What I set out to do

Entry 05 built the products *data layer* — the `useProducts`, `useCreateProduct`, etc. hooks — but no component actually used them. This session I finally connected the UI to real data:

1. `ProductList` — show all products in a table, with Edit/View/Delete actions
2. `ProductForm` **(view mode)** — fetch and display a single product
3. Added the missing `useGetProduct(id)` hook the view/edit pages needed

And I hit **two real bugs** along the way (both things I asked "what's wrong?" about), which — per my own rule — are the most valuable part of this entry.

Part 1 — The product list (`ProductList.tsx`)

This is my first component that reads real server data:

```tsx
import { Link } from "@tanstack/react-router"
import { useDeleteProduct, useProducts } from "../product.query"

export const ProductList = () => {
  const { data: products, isLoading, isError, error } = useProducts()
  const deleteProduct = useDeleteProduct();

  const onDelete = (id: number) => {
    deleteProduct.mutate(id)
  }

  return (
    <>
      <h1>Product List</h1>
      <Link to="/products/add">Add Product</Link>

      <table>
        {/* ...thead... */}
        <tbody>
          {products?.map((product, index) => (
            <tr key={product.id}>
              <td>{index + 1}</td>
              <td>{product.name}</td>
              <td>{product.price}</td>
              <td>{product.description}</td>
              <td>
                <Link to="/products/$productId/edit" params={{ productId:
String(product.id) }}>Edit</Link>
                <Link to="/products/$productId" params={{ productId: String(product.id)
}}>View</Link>
                <button onClick={() => onDelete(product.id)}>Delete</button>
              </td>
            </tr>
          ))}
        </tbody>
      </table>

      {isLoading && <h2>Loading...</h2>}
      {isError && <h3>{error.message}</h3>}
    </>
  )
}
```

Things I learned wiring this up:

- `data: products` — that colon is *renaming* while destructuring. `useProducts` returns a field literally called `data`; I rename it to `products` so the JSX reads nicely. (`isLoading`, `isError`, `error` come straight from the same hook — all the loading/error bookkeeping I got for free from TanStack Query, entry 05.)
- `products?.map(...)` — the `?.` (optional chaining) matters. On the very first render, before the fetch finishes, `products` is `undefined`, and `undefined.map()` would crash. `?.` means "only call `.map` if `products` exists, otherwise give `undefined`" (which React renders as nothing).
- `key={product.id}` — same lesson as the todo list (entry 03): stable id as the key, not the array index.
- **Typed links with `params`** — the Edit/View links use the `$productId` dynamic route from entry 02. I don't build the URL string by hand; I give the route pattern plus `params`, and because the route ids are numbers but URLs are strings, I convert with `String(product.id)`.
- **Delete = a mutation.** `useDeleteProduct()` gives me a mutation object, and `deleteProduct.mutate(id)` fires it. Thanks to the `invalidateQueries` in that hook (entry 05), the list refreshes itself automatically after a delete — I didn't write a single line to remove the row manually.

🐛 Bug I hit #1: rendering an error object

My first version of the error line was:

```
// ❌ WRONG — my first attempt
{isError && <h3>{error}</h3>}
```

TypeScript complained: *"Type 'Error' is not assignable to type 'ReactNode'"*, and at runtime React would throw *"Objects are not valid as a React child"*. The problem: `error` is an `Error` **object**, not a string — and React can't render an object. The human-readable text lives in its `.message` property:

```
// ✅ RIGHT
{isError && <h3>{error.message}</h3>}
```

(A nice detail: because `useQuery`'s result is a *discriminated union*, inside the `isError &&` block TypeScript already knows `error` is a real `Error` and not `null`, so I can safely reach `.message`.)

Part 2 — The `useGetProduct` hook

The view and edit pages need to fetch **one** product by id. The API function `getProduct(id)` existed since entry 05, but there was no hook wrapping it. I added:

```ts
export const productKeys = {
    all: ['products'] as const,
    detail: (id: number) => ['product', id] as const,
};

export const useGetProduct = (id: number) => {
    return useQuery({
        queryKey: productKeys.detail(id),
        queryFn: () => getProduct(id)
    });
};
```

🐛 Bug I hit #2: how a query gets its argument

My first attempt looked reasonable but was wrong in three ways at once:

```ts
// ❌ WRONG — my first attempt
export const productKeys = {
    detail: ['product'] as const,        // static key, same for every product
};

export const useGetProduct = () => {     // no id parameter!
    return useQuery({
        queryKey: productKeys.detail,
        queryFn: (id: number) => getProduct(id),   // this id is NOT what I think
    });
}
```

What's wrong, and why:

1. **A query's `queryFn` does not receive my `id`.** This is the big misunderstanding. Unlike a *mutation*, where `mutate(id)` passes the value into `mutationFn` (entry 05), a **query runs automatically** based on its key — nobody "calls it" with an argument. The function TanStack Query actually passes to `queryFn` is a *context object* (with `queryKey`, `signal`, …), so my `id: number` would really be that object, and I'd fetch `/products/[object Object]`.
2. **The hook took no `id`.** There was nowhere for the id to enter. It has to be a parameter of the hook, and the `queryFn` grabs it by **closure**, not as an argument.
3. **The key was shared across all products.** `['product']` is identical for product 1, 5, and 50 — so they'd all collide in one cache slot. View product 1, then product 2, and you'd get product 1's stale data. The key **must include the id**.

The fix addresses all three: the hook takes `id`, `detail` becomes a *function* that folds the id into the key (`['product', 5]` — unique per product), and `queryFn` reads `id` from the closure:

```
// ✅ RIGHT
detail: (id: number) => ['product', id] as const,

export const useGetProduct = (id: number) => useQuery({
    queryKey: productKeys.detail(id),
    queryFn: () => getProduct(id),
});
```

The mental model I'm taking away: *mutations are called with arguments; queries are described by a key and run themselves.*

Part 3 — ProductForm view mode

`ProductForm` now fetches the product when showing it:

```
export const ProductForm = ({ mode, productId }: ProductFormProps) => {
  const { data, isLoading, isError } = useGetProduct(Number(productId));

  const [name, setName] = useState<string>("");
  const [price, setPrice] = useState<string>("");
  const [description, setDescription] = useState<string>("");

  if (mode === 'view') {
    return (
      <>
        {isLoading && <h1>Loading...</h1>}
        {isError && "Error has occured"}
        {data && (
          <>
            <h3>Name: {data.name}</h3>
            <h3>Price: {data.price}</h3>
            <h3>Description: {data.description}</h3>
          </>
        )}
      </>
    )
  }

  // add / edit → the controlled form inputs
  return (
    <>
      <div><label>Name</label><input value={name} onChange={(e) => setName(e.target.value)}
/></div>
      <div><label>Price</label><input value={price} onChange={(e) =>
setPrice(e.target.value)} /></div>
      <div><label>Description</label><input value={description} onChange={(e) =>
setDescription(e.target.value)} /></div>
    </>
  )
}
```

- `Number(productId)` — remember from entry 02 that route params arrive as **strings**, but `useGetProduct` and the API expect a **number**, so I convert.
- **View mode** uses the fetched `data`; **add/edit mode** shows controlled inputs (the `value` + `onChange` pattern from the todo list, entry 03).
- `{data && (...)}` guards the display so it only renders once the data has arrived — same idea as `products?.map` above.

Things that tripped me up / notes to self

- **Queries vs mutations, again.** The `useGetProduct` bug was really me applying mutation thinking to a query. Worth burning in: queries fetch based on a key, automatically; mutations are fired manually with an argument.
- ⚠️ **The form isn't actually saving yet.** The add/edit inputs have state but there's **no submit button** and they're **not wired to `useCreateProduct` / `useUpdateProduct`**. Also, in *edit* mode the inputs start empty — they don't pre-fill from the fetched product. So edit mode currently can't really edit. Next session's job.
- ⚠️ **`useGetProduct` fires even in add mode.** When `mode === 'add'`, `productId` is `undefined`, so `Number(undefined)` is `NaN`, and the hook still runs a query for `/products/NaN` (hooks must run unconditionally, so I can't just skip it). That's a wasted, failing request. The fix is TanStack Query's `enabled` option, e.g. `enabled: mode === 'view' || mode === 'edit'`, so it only fetches when there's a real id.
- ⚠️ **Price is a string in state.** `useState<string>("")` for price is fine for the input, but I'll need `Number(price)` before sending it to the API (the server and `IProduct` expect a number).
- **Minor:** view-mode error just shows the literal `"Error has occured"` (and it's spelled "occurred"). Fine as a placeholder, but it throws away `error.message`.
- **`isLoading` vs `isPending`.** Works today; note that v5's primary "no data yet" flag is `isPending`. Same behavior on first load.

What's next

- [ ] Add a submit button and wire add mode to `useCreateProduct`
- [ ] Pre-fill the edit form from `useGetProduct` data, wire to `useUpdateProduct`
- [ ] Add `enabled` to `useGetProduct` so it doesn't fetch in add mode
- [ ] Convert `price` to a number before submitting
- [ ] Show the server's real error message instead of a hardcoded string
- [ ] A confirm dialog + disabled state (`deleteProduct.isPending`) on Delete

CHAPTER 8

Making the Form Actually Save: Create & Edit Products

2026-07-14

What I set out to do

Entry 07 left `ProductForm` half-finished: it could *show* a product (view mode) and it had input boxes for add/edit, but the boxes did nothing — no Save button wired up, the edit form didn't pre-fill, and it even fired a pointless request in add mode. This session I closed all of that out. By the end, the form can genuinely **create** a new product and **edit** an existing one, then send me back to the list.

This entry basically works through entry 07's "What's next" checklist:

- [x] Only fetch in edit/view mode (not add)
- [x] Pre-fill the edit form from the fetched product
- [x] Wire Save to `useCreateProduct` / `useUpdateProduct`
- [x] Convert `price` to a number before sending

Part 1 — Only fetch when there's actually a product (`enabled`)

In entry 07 I flagged a bug: in **add** mode there's no `productId`, so `Number(undefined)` is `NaN`, and the fetch hook still ran — firing a doomed request for `/products/NaN`. The fix is TanStack Query's `enabled` option, which I threaded through my hook:

```
// product.query.ts
export const useGetProduct = (id: number, enabled: boolean) => {
    return useQuery({
        queryKey: productKeys.detail(id),
        queryFn: () => getProduct(id),
        enabled                          // when false, the query stays idle — no request
    });
};
```

And in the form I pass `mode !== 'add'`:

```
const { data, isLoading, isError } = useGetProduct(Number(productId), mode !== 'add');
```

So view and edit fetch; add stays quiet. `enabled: false` means the query just sits there in an idle state and never touches the network — exactly what I want when there's nothing to load yet. (This is the same option you'd use for "don't search until the user has typed something.")

Part 2 — Pre-filling the edit form (`useEffect`)

The other entry-07 gap: in edit mode the inputs started empty instead of showing the product's current values. The problem is timing — my inputs are controlled by `useState` starting at `""`, but the fetched `data` arrives *later*, asynchronously. On the first render `data` is still `undefined`, so I can't seed `useState` from it.

The tool for "do something when async data arrives" is `useEffect`:

```
useEffect(() => {
  if (data) {
    setName(data.name);
    setPrice(String(data.price));     // price is a number in data, but state is string
    setDescription(data.description);
  }
}, [data]);                           // re-run whenever `data` changes
```

How it plays out:

- First render: `data` is `undefined`, the `if (data)` guard fails, nothing happens.
- Fetch resolves: `data` becomes the product, its reference changes, the effect re-runs (because `data` is in the dependency array `[data]`), and the three fields fill in.
- Add mode: the query is disabled, `data` stays `undefined`, so the effect never fills anything — inputs stay empty. Perfect.

⚠️ One thing I now understand is a *known* trade-off, not a bug: this copies server data into local state, so if the product ever refetched in the background, the effect would overwrite whatever I'd typed. Fine for this app, but "server data → form state" is a genuinely fiddly area in bigger apps.

> Why not just call `setName(data.name)` directly in the component body instead of an effect? Because calling a state setter during render triggers another render, which sets state again → an infinite loop. React literally warns about this. Copying external data into state is what `useEffect` is *for*.

Part 3 — The Save handler (the main event)

I call both mutation hooks (from entry 05) near the top — hooks must run unconditionally, and they don't do anything until I call `.mutate()`:

```
import { useNavigate } from "@tanstack/react-router"

const navigate = useNavigate();
const createProduct = useCreateProduct();
const updateProduct = useUpdateProduct();
```

Then `handleSave` decides which mutation to fire based on `mode`:

```
const handleSave = () => {
  if (!name || !price) return;                    // guard: don't save empty

  const body = {
    id: Number(productId),        // ignored by the server on create; used on update
    name,
    price: Number(price),         // state is a string → convert to number for the API
    description,
  };

  if (mode === 'edit') {
    updateProduct.mutate(
      { id: Number(productId), body },
      { onSuccess: () => navigate({ to: '/products' }) },
    );
  } else {
    createProduct.mutate(
      body,
      { onSuccess: () => navigate({ to: '/products' }) },
    );
  }
};
```

The pieces that clicked for me:

- **`Number(price)`** — the input state is a string (`useState<string>`), but the API and `IProduct` want a number. Same string↔number seam as the route params from entry 02 and the pre-fill above. I keep meeting it.
- **The two mutations take different arguments.** `updateProduct.mutate` wants `{ id, body }` (that's the `mutationFn: ({ id, body }) => ...` shape from entry 05), while `createProduct.mutate` takes the body directly. That asymmetry is why the branches look different.
- **The second argument to `.mutate(...)` is per-call options.** Its `onSuccess` runs *after this specific save succeeds* — I use it to `navigate` back to the list. This runs **in addition to** the

`onSuccess` already inside the hook (which does `invalidateQueries`). So on a successful save, two things happen: the cache is invalidated *and* I get redirected.

Why the list is already up-to-date when I land

This is the part I find genuinely satisfying. I don't manually add the new product to any list. Because `useCreateProduct` / `useUpdateProduct` call `invalidateQueries({ queryKey: productKeys.all })` on success (entry 05), the moment `navigate({ to: '/products' })` lands me on the list, TanStack Query sees the `['products']` cache is stale and refetches it — so my new/edited product is just *there*. The form page and the list page never talk to each other directly; they're connected only through the query key.

The full loop:

```
handleSave → mutate → server saves → hook onSuccess invalidates ['products']
           → per-call onSuccess navigates to /products → list refetches → fresh data
```

Part 4 — `useNavigate` for the redirect

```tsx
const navigate = useNavigate();
// ...
navigate({ to: '/products' });
```

`useNavigate` is TanStack Router's hook for moving around **in code** (as opposed to a `<Link>` the user clicks — entry 07). Same type-safety perk as links: if I typo `/product` it won't compile. I call it inside `onSuccess` so the redirect only happens *after* the save actually succeeds — never navigate away from a save that failed.

Things that tripped me up / notes to self

- **Mutations vs queries, one more time.** Save is a mutation: I *call* it with an argument (`.mutate(body)`). Fetching is a query: it runs itself from a key. Getting this straight (after the entry 07 bug) made this whole session smooth.
- ⚠️ **Editing doesn't refresh the single-product cache.** `useUpdateProduct` invalidates `['products']` (the list) but not `['product', id]` (the detail). So right after editing, a View might show stale data until it goes stale on its own. Fix: also invalidate `productKeys.detail(id)` in the update hook's `onSuccess` .
- ⚠️ **No "saving..." state or disabled button.** A fast double-click could fire two saves. `disabled={createProduct.isPending || updateProduct.isPending}` on the button would fix it. Left out for now.

- ⚠️ **Weak validation.** I only check `name` and `price` are non-empty. If I type "abc" into price, `Number("abc")` is `NaN` and I'd send junk. Real validation (Zod?) is a future topic.
- ⚠️ **The `id` in the create body is still a white lie** — the server generates the id and ignores mine; it's only there to satisfy the `IProduct` type. `Omit<IProduct, 'id'>` remains the honest fix (first noted in entry 05).
- **Leftover typo:** view mode still says "Error has occured" (should be "occurred") and throws away the real `error.message`.

What's next

- [ ] Invalidate `productKeys.detail(id)` after an edit
- [ ] Disable Save while `isPending`, show a "Saving..." state
- [ ] Proper form validation (numeric price, required fields) — maybe with Zod
- [ ] Switch the create body type to `Omit<IProduct, 'id'>`
- [ ] Show a success message / toast instead of a silent redirect

CHAPTER 9

Real Login Authentication (Token, Guard & Interceptor)

2026-07-15

What I set out to do

Up to now, being "logged in" was faked in three places — `isAuthenticated = true` hardcoded in the route guard, a `"dummy"` token in the axios client, and a login page that did nothing. This session I replaced all three fakes with a **real login flow**:

1. Type email + password → call `POST /auth/login`
2. Get a token back → store it in `localStorage`
3. Send that token on every API request (the interceptor from entry 04, for real)
4. The route guard (entry 01) now checks for a real token, not a hardcoded `true`

By the end, logging in actually logs me in, and visiting a protected page without a token bounces me to `/login`.

Part 1 — The auth feature files

I made a new `src/features/auth/` feature, same shape as products (entry 02): types, an api function, a query hook, a component, and a barrel.

The types (`auth.types.ts`)

```
export interface ILoginBody {
    email: string;
    password: string
}

export interface ILoginResponse {
    token: string;
    user: any;
}
```

`ILoginBody` is what I *send*; `ILoginResponse` is what the server *returns* — a token and a user. (⚠️ `user: any` is a placeholder — `any` turns off type-checking for that field. I'll give it a real `IUser` shape once I actually use the user data.)

The API call (`auth.api.ts`)

```
import { apiClient } from '@/lib'
import type { ILoginBody, ILoginResponse } from './auth.types';

export const login = async (body: ILoginBody) => {
  const res = await apiClient.post<ILoginResponse>('/auth/login', body)
  return res.data
}
```

Exactly the pattern from entry 05's products API — a thin typed wrapper that returns `res.data`. Because it goes through `apiClient`, it automatically gets the base URL and timeout. (It doesn't need a token itself — logging in is how you *get* the token.)

The hook (`auth.query.ts`)

```
import { useMutation } from '@tanstack/react-query'
import { login } from './auth.api'

export const useLogin = () => useMutation({ mutationFn: login })
```

Logging in is a **mutation**, not a query — it's an action I trigger, not data I read (same reasoning as saving a product in entry 08). Notice there's no `onSuccess` with `invalidateQueries` here like the product mutations had — logging in doesn't make any cached list stale, so there's nothing to invalidate.

Part 2 — The login page (`components/LoginPage.tsx`)

```
export const LoginPage = () => {
    const login = useLogin();
    const navigate = useNavigate();

    const [email, setEmail] = useState('');
    const [password, setPassword] = useState('');

    const handleLogin = () => {
        login.mutate(
            { email, password },
            {
                onSuccess: (data) => {
                    localStorage.setItem('token', data.token);
                    navigate({ to: '/products' });
                },
            },
        )
    }

    return (
        <>
            <h1>Login</h1>
            <div>
                <label>Username</label>
                <input value={email} onChange={(e) => setEmail(e.target.value)} type="text"
/>
            </div>
            <div>
                <label>Password</label>
                <input value={password} onChange={(e) => setPassword(e.target.value)}
type="password" />
            </div>
            <button onClick={handleLogin} disabled={login.isPending}>Login</button>
            {login.isError && login.error.message}
        </>
    )
}
```

The moving parts:

- **Controlled inputs** (entry 03) — `value` + `onChange` — are what let `handleLogin` actually read what was typed. Without them the inputs would be uncontrolled and I couldn't get the values.
- `login.mutate({ email, password }, { onSuccess })` — fire the mutation with the form values, and in the per-call `onSuccess` (entry 08) do the two things that "being logged in" means: save the token, then navigate into the app.
- `localStorage.setItem('token', data.token)` — `localStorage` is browser storage that survives refreshes and tab closes (unlike a React state variable, which resets on reload). That persistence is the whole point — I stay logged in across refreshes.
- `disabled={login.isPending}` — the button disables itself while the request is in flight, so I can't double-submit. (This is the pattern I said I'd add for the product form back in entry 08, applied here from the start.)
- `{login.isError && login.error.message}` — show the error if login fails.

Then the route file just points at it (thin routes, entry 02):

```
// src/routes/login.tsx
import { LoginPage } from '@/features/auth'
export const Route = createFileRoute('/login')({ component: LoginPage })
```

Part 3 — Using the real token everywhere

Two placeholders finally became real.

The axios interceptor (`apiClient.ts`)

```
 apiClient.interceptors.request.use((requestConfig) => {
-  const token = "dummy";
+  const token = localStorage.getItem('token');
   if (token) {
     requestConfig.headers.Authorization = `Bearer ${token}`;
   }
   return requestConfig;
 });
```

This is the entry-04 request interceptor doing its real job now: before every request leaves, it reads the stored token and attaches it as a `Bearer` header. Every product call I've made is now authenticated as the logged-in user, automatically — I never touch headers in my API functions.

The route guard (`_authenticated.tsx`)

```
-//temporary
-const isAuthenticated = true;
-
 export const Route = createFileRoute('/_authenticated')({
   beforeLoad: ({ location }) => {
+    const isAuthenticated = !!localStorage.getItem('token');
     if (!isAuthenticated) {
       throw redirect({ to: '/login', search: { redirect: location.href } })
     }
   },
```

The "bouncer at the door" from entry 01 now checks a real credential — is there a token in `localStorage`? — instead of always waving everyone through. (`!!` turns the value into a boolean: a real string → `true` , `null` → `false` .)

🐛 Bug I hit: the redirect that bounced me right back

While building this, login *looked* broken: I'd log in, the token was clearly being saved, but I'd immediately end up back on the login page. My first version of the guard put the check at the **top of the module**:

```
// ❌ WRONG — my first attempt
const isAuthenticated = !!localStorage.getItem('token');   // module level!

export const Route = createFileRoute('/_authenticated')({
  beforeLoad: ({ location }) => {
    if (!isAuthenticated) { throw redirect({ to: '/login', ... }) }
  },
  ...
})
```

Why it fails: module-level code runs **once**, when the file is first imported at app startup. At startup there's no token, so `isAuthenticated` is captured as `false` and frozen forever. So after logging in:

1. Token gets saved ✅
2. `navigate({ to: '/products' })` fires ✅
3. `/products` is protected, so `beforeLoad` runs
4. It checks the **stale** `false` from startup — not a fresh read
5. → throws redirect back to `/login`

The navigate worked; the guard was just looking at a snapshot from before I logged in and kicking me out again.

```
// ✅ RIGHT — read inside beforeLoad, so it runs fresh every navigation
beforeLoad: ({ location }) => {
  const isAuthenticated = !!localStorage.getItem('token');
  if (!isAuthenticated) { throw redirect({ to: '/login', ... }) }
}
```

The lesson: *module-level code runs once at import; function-body code runs every time the function is called.* Anything that changes over the app's life — like "am I logged in?" — must be read **inside** the function that needs it. `beforeLoad` runs on every navigation, so the check has to live there.

Things that tripped me up / notes to self

- **Once vs every time.** The core lesson from the bug above — the single most useful thing I learned this session.
- ⚠️ **Logout isn't wired up.** There's no logout button, and the response interceptor still redirects on 403 **without clearing the token** (the old `//todo logout func`). So an expired token would loop: redirect to login, but the stale token's still there. Logout needs to `localStorage.removeItem('token')`.
- ⚠️ **I built a `?redirect=` param but don't use it.** The guard saves where I was headed into `search.redirect`, but `LoginPage` always navigates to `/products`, ignoring it. Next step: read that param and go back there after login.
- ⚠️ `user: any`**.** No real type on the returned user yet, and I'm not even storing the user — only the token. Authorization (roles) will need the user object.
- ⚠️ **Security note (localStorage).** A token in `localStorage` is readable by any JS on the page, so it's XSS-exposed. Fine for learning; production would lean toward an httpOnly cookie (and `apiClient` already has `withCredentials: true` for that day).
- **Small stuff:** the label says "Username" but it's really an email; and `isError` shows axios's generic `"Request failed with status code 401"`, not the server's nicer `"Invalid email or password"` message.
- Reminder of the seeded login: `sandip@example.com` / `password123`.

What's next

- [ ] Logout: clear the token + redirect, and clear it in the 403 interceptor too
- [ ] After login, honor the `?redirect=` param instead of always going to `/products`
- [ ] Store the `user` too, give it a real `IUser` type (drop the `any`)
- [ ] Authorization: use `user.role` to gate admin-only UI and routes
- [ ] Show the server's real error message on failed login

CHAPTER 10

Global State with Zustand: An Auth Store

2026-07-15

What I set out to do

After entry 09, my auth state worked but was **scattered**: I read `localStorage.getItem('token')` directly in three different places (the axios client, the route guard, and the login page set it). If I ever changed how auth worked, I'd have to hunt down every spot. This session I learned **Zustand** — a tiny global-state library — and used it to pull all that auth state into **one store**. As a bonus, it knocked out two leftovers from entry 09: storing the `user` (not just the token) and having a real `logout`.

The big idea: client state vs server state

This was the concept I had to get straight before writing a line, because my app *already* has a state library — TanStack Query (entries 04–08). Don't they overlap? No — they're for **two different kinds of state**:

	Server state → TanStack Query	**Client state** → Zustand
What	Data the server owns	Data only the browser cares about
In my app	products, single product	**auth token, logged-in user**
Source of truth	the backend	the app itself

⚠️ **The trap I now know to avoid:** don't copy fetched data (like products) into Zustand. TanStack Query already caches and refetches that (entry 05). Auth is a perfect fit for Zustand because the *token* isn't server data I re-fetch — it's session state the app holds onto.

Part 1 — Installing & building the store

`npm install zustand` (it's now in `package.json`), then a new file:

```ts
// src/features/auth/auth.store.ts
import { create } from 'zustand'
import { persist } from 'zustand/middleware'

interface AuthState {
  token: string | null
  user: { id: number; name: string; email: string } | null
  setAuth: (token: string, user: AuthState['user']) => void
  logout: () => void
}

export const useAuthStore = create<AuthState>()(
  persist(
    (set) => ({
      token: null,
      user: null,
      setAuth: (token, user) => set({ token, user }),
      logout: () => set({ token: null, user: null }),
    }),
    { name: 'auth' },   // localStorage key
  ),
)
```

Unpacking the new stuff:

- **`create(...)`** makes the store. The function it takes receives **`set`** (the updater, just like `setState`) and returns the initial state **plus actions** — functions that call `set` to change things. So the state (`token`, `user`) and the logic that changes it (`setAuth`, `logout`) live together in one place. That co-location is the thing I was missing when auth was scattered.
- **`persist(...)`** is *middleware* — a wrapper that adds behavior. This one automatically saves the store to `localStorage` (under the key `"auth"`) and reloads it on refresh. So I get persistence for free, and it **replaces my manual `localStorage.setItem/getItem`** from entry 09. And now the `user` is saved too, not just the token.
- The whole thing is exported as **`useAuthStore`** — it's both a hook *and* an object with extra methods (more on that below).

Part 2 — Reading the store *inside* a component (selectors)

In `LoginPage`, the manual `localStorage.setItem` became a store action:

```
const setAuth = useAuthStore((s) => s.setAuth);   // grab the action at the top

const handleLogin = () => {
  login.mutate({ email, password }, {
    onSuccess: (data) => {
      setAuth(data.token, data.user);   // save token + user in the store
      navigate({ to: '/products' });
    },
  })
}
```

- `useAuthStore((s) => s.setAuth)` — that `(s) => s.setAuth` is a **selector**: it picks out just the one slice I need. The component then only re-renders when *that slice* changes (grabbing the whole store would re-render on every change to any field). Selectors are the key habit for keeping Zustand fast.
- **Why grab it at the top, not inside `onSuccess`?** Because `useAuthStore(...)` is a hook, and hooks must be called at the **top level of a component** — never inside callbacks or conditions (the Rules of Hooks). So I select the action at the top, then *call* it (`setAuth(...)`) later inside the callback. Selecting and calling are two separate steps.

Part 3 — Reading the store *outside* a component (`.getState()`)

Here's the part that took a real "aha." The axios interceptor and the router's `beforeLoad` are **not React components** — they're plain functions — so I **cannot** call the `useAuthStore(...)` hook in them (hooks only work inside components). Zustand's answer: every store also has a plain `.getState()` method for non-React access.

```
// apiClient.ts — request interceptor
apiClient.interceptors.request.use((requestConfig) => {
  const token = useAuthStore.getState().token;   // no hook — just read the value
  if (token) requestConfig.headers.Authorization = `Bearer ${token}`;
  return requestConfig;
});
```

```
// _authenticated.tsx — the route guard
beforeLoad: ({ location }) => {
  const isAuthenticated = !!useAuthStore.getState().token;   // fresh read each navigation
  if (!isAuthenticated) {
    throw redirect({ to: '/login', search: { redirect: location.href } })
  }
}
```

Two things I like about this:

- `.getState()` reads the value **fresh at call time** — which is exactly what fixed my entry-09 redirect bug (read inside the function, not once at module load). The store makes the correct pattern the natural one.
- The guard is still the "bouncer at the door" from entry 01 — it just checks the store now instead of poking at `localStorage` directly.

The rule, boiled down

> **Hook + selector inside components** (`useAuthStore(s => s.x)`). **`.getState()` everywhere else** (interceptors, `beforeLoad` , plain functions).

🐛 Bug I hit: selecting `logout` but never logging out

I wired `logout` into the response interceptor (so an expired 403 session clears itself). My first attempt was wrong in **two** ways at once:

```
// ❌ WRONG — my first attempt
if (error.response?.status === 403) {
  useAuthStore((s) => s.logout)      // two bugs on one line
  window.location.href = '/login';
}
```

1. **Hook outside a component.** The interceptor isn't a component, so calling the `useAuthStore(...)` hook here is illegal — React throws "invalid hook call." This needs `.getState()` .
2. **Selected but never called.** Even ignoring bug 1, `(s) => s.logout` just *grabs a reference* to the function and throws it away — there are no `()` , so it never runs.

The fix does both correctly — read outside React **and** actually call it:

```
// ✅ RIGHT
if (error.response?.status === 403) {
  useAuthStore.getState().logout();   // .getState() = no hook; () = actually run it
  window.location.href = '/login';
}
```

Now a 403 wipes `token` + `user` from the store (and, thanks to `persist`, from localStorage) *before* redirecting — fixing the "stale token loops forever" problem I flagged in entry 09. The `window.location.href` (full reload, not `navigate`) is actually good right after logout: it guarantees all in-memory state and the TanStack Query cache are wiped clean.

The lesson: the difference between `LoginPage` and the interceptor isn't style — it's that one is a component (hooks legal) and one isn't (`.getState()` required). And selecting a function is not the same as calling it.

What this cleaned up

Before, "am I logged in?" was answered by three separate `localStorage` pokes. Now there's **one store** and everything reads from it:

Place	Before (entry 09)	After (Zustand)
Login success	`localStorage.setItem('token', ...)`	`setAuth(token, user)`
API requests	`localStorage.getItem('token')`	`useAuthStore.getState().token`
Route guard	`localStorage.getItem('token')`	`useAuthStore.getState().token`
403 / logout	*(nothing — bug!)*	`useAuthStore.getState().logout()`

One source of truth, and the `user` is finally stored, not just the token.

Things that tripped me up / notes to self

- **Component vs not-a-component decides hook vs `.getState()`.** The single biggest takeaway, and the root of the logout bug.
- **Select ≠ call.** `(s) ⇒ s.logout` hands me the function; `logout()` runs it.
- ⚠️ **No logout *button* yet.** The `logout` action exists and fires on a 403, but there's no button for the user to log out on purpose. Easy to add now: `const logout = useAuthStore(s ⇒ s.logout)` then call it + navigate.
- ⚠️ **`?redirect=` still unused.** `LoginPage` always goes to `/products`, ignoring the param the guard saves. Still on the list from entry 09.
- ⚠️ **`user`'s type is duplicated.** The store types `user` inline as `{ id; name; email }`, but `ILoginResponse.user` is still `any` (entry 09). So `setAuth(data.token, data.user)` passes `any`

into a typed slot — it compiles, but the source type is loose. Worth a shared `IUser` type.

- ⚠️ **Security, still.** `persist` uses `localStorage` by default, so the same XSS caveat from entry 09 applies. Fine for learning.
- **Storage key changed.** Auth now lives under the `"auth"` key as JSON (`{ state: { token, user }, version }`), not the old raw `"token"` string. An old `"token"` entry from before could linger harmlessly in my browser.

What's next

- [ ] Add a visible Logout button (in `AuthenticatedLayout` ?)
- [ ] Honor `?redirect=` after login instead of always `/products`
- [ ] Give `user` a shared `IUser` type; drop the `any` in `ILoginResponse`
- [ ] Authorization: read `user.role` from the store to gate admin-only UI/routes
- [ ] (Maybe) select `user` in a component to show "Logged in as {name}"

CHAPTER 11

React Hook Form + Zod in the Product Form

2026-07-16

What I set out to do

My `ProductForm` worked (entry 08), but validation was hand-rolled — a weak `if (!name || !price) return`, three separate `useState`s, a `useEffect` to pre-fill, and manual `Number(price)` conversions. This session I replaced all of that with two libraries that are made for exactly this:

- **React Hook Form (RHF)** — manages the form's values, submission, and field states (touched, dirty, errors) for me.
- **Zod** — describes the validation rules as a schema (I already met Zod validating env vars in entry 04).

They connect through a tiny bridge package, `@hookform/resolvers`.

Part 1 — Installing

```
npm install react-hook-form @hookform/resolvers
```

`react-hook-form` is the form library; `@hookform/resolvers` is the adapter that lets RHF use a Zod schema for validation. (Zod itself was already a dependency.)

Part 2 — The schema is the single source of truth

```
import { z } from 'zod'

const productSchema = z.object({
  name: z.string().min(1, 'Name is required'),
  price: z.number().positive('Price must be greater than 0'),
  description: z.string().min(1, 'Description is required'),
})

type ProductFormValues = z.infer<typeof productSchema>
```

Two things I love here:

- The validation messages (`'Name is required'`) live **in the schema**, right next to the rule they belong to.

- `z.infer<typeof productSchema>` turns the schema *into* a TypeScript type. So I define the rules once and get `{ name: string; price: number; description: string }` for free — no separate interface to keep in sync. Schema and type can never drift.

Part 3 — Wiring up `useForm`

```
const {
  register,
  handleSubmit,
  reset,
  formState: { errors, touchedFields, dirtyFields },
} = useForm<ProductFormValues>({
  resolver: zodResolver(productSchema),
  defaultValues: { name: '', price: 0, description: '' },
  mode: 'onTouched',
});
```

- `resolver: zodResolver(productSchema)` — this is the bridge. It tells RHF "run everything through this Zod schema to decide what's valid."
- `defaultValues` — the form's starting values.
- `mode: 'onTouched'` — *when* validation runs (see the touched/dirty bug below).

Then the inputs change completely. My old inputs were **controlled** (entry 03) — `value={name}` `onChange={...}`, with React state behind each one. RHF inputs are **uncontrolled**: I just spread `register('name')` onto the input, and RHF wires up the value/onChange/onBlur/ref internally by reading the DOM through a **ref** (a direct handle to the DOM element). No `useState` per field.

```
<input type="text" {...register('name')} />
{(touchedFields.name || dirtyFields.name) && errors.name && <p>{errors.name.message}</p>}
```

And submission:

```
<form onSubmit={handleSubmit(onSubmit)}>
```

`handleSubmit(onSubmit)` runs the Zod schema first. If validation fails, it fills `errors` and my `onSubmit` **never runs**. If it passes, `onSubmit` gets clean, typed values. That single mechanism replaced my old `if (!name || !price) return` guard.

What disappeared

- 3 × `useState` → RHF holds the values

- manual pre-fill `setName/setPrice/setDescription` → **`reset(data)`** loads all values at once (still in a `useEffect` on `[data, reset]`, same async-timing reason as entry

  8.

- `Number(price)` everywhere → the schema handles the number (see the bug below)
- manual empty-check → `handleSubmit` + schema
- as a bonus, it's a real `<form>`, so **Enter submits**

I kept **view mode exactly as it was** — it's not a form, so RHF doesn't touch it.

🐛 Bug I hit #1: `zod` wasn't actually installed

The moment I imported `z`, TypeScript errored: *"Cannot find module 'zod'."* But zod was right there in `package.json`, and `env.ts` had been importing it for ages! Running a typecheck showed `env.ts` was *also* silently broken.

The cause is the **phantom dependency** I first flagged back in entry 04: zod was in `package.json` but never truly installed — it had only existed in `node_modules` *transitively* (pulled in by some other package), and that provider was now gone. It "worked" by luck until I leaned on it directly.

The fix was simply to install it for real:

```
npm install zod
```

Lesson: a package must be a real, installed dependency — "it happens to be in node_modules" is not the same thing. This is exactly the trap the entry-04 notes warned about, and it finally bit me.

🐛 Bug I hit #2: `z.coerce.number()` broke the types

My first schema used `z.coerce.number()` (to auto-convert the input's string to a number). The typecheck then complained that the resolver's types didn't match `useForm`:

```
Type 'unknown' is not assignable to type 'number'
```

Why: `z.coerce.number()` accepts *`unknown`* as input (it'll coerce anything) and outputs a `number`. So the schema's **input** type and **output** type differ, and RHF got confused about which one the form values are.

The fix — keep the schema as a plain `z.number()`, and let RHF do the string→number conversion at the input with `valueAsNumber`:

```
// schema
price: z.number().positive('Price must be greater than 0'),

// input
<input type="number" {...register('price', { valueAsNumber: true })} />
```

`valueAsNumber: true` tells RHF to hand the field to the schema as a number, so input and output types line up. Cleaner than juggling coerce, and it kills the manual `Number(price)` I used to do.

🐛 Bug I hit #3: the touched/dirty error wouldn't show (`||` vs `&&`)

I wanted the name error to appear only after the field was touched or edited (like Angular's `(touched || dirty) && invalid`). My first attempt:

```
// ❌ WRONG
{(touchedFields.name || dirtyFields.name) || errors.name && <p>{errors.name.message}</p>}
```

Why it fails — operator precedence. `&&` binds tighter than `||`, so JS read it as:

```
{(touchedFields.name || dirtyFields.name)  ||  (errors.name && <p>...</p>)}
```

The moment I touched the field, the left side became `true`, the `||` short-circuited, and the whole thing returned `true` (which React renders as nothing) — the `<p>` was never reached. So the error could only show *before* touching, the opposite of what I wanted.

```
// ✅ RIGHT
{(touchedFields.name || dirtyFields.name) && errors.name && <p>{errors.name.message}</p>}
```

One character: the middle `||` becomes `&&`. Now it reads "if (touched or dirty) **and** invalid, show the error."

The other half of that fix: `mode: 'onTouched'`

Even after fixing the operator, the error still didn't show while typing — because RHF's default `mode` is `'onSubmit'`, so `errors` stays empty until the first Save. I added `mode: 'onTouched'` so validation runs on blur (then re-checks as I type). The validation modes:

`mode`	When it validates
`'onSubmit'` (default)	only when Save is clicked
`'onBlur'`	when a field loses focus
`'onTouched'`	first on blur, then on every change ← Angular-like
`'onChange'`	every keystroke

Things that tripped me up / notes to self

- **Controlled vs uncontrolled.** My todo/login forms are controlled (entry 03); RHF is uncontrolled (reads via refs). Both are valid — good to know the difference and why RHF is fast (fewer re-renders).
- **Phantom dependencies are real.** `npm install zod` should have happened long ago.
- `z.infer` **= schema becomes the type.** The pattern I'll reuse everywhere.
- `mode` **controls *when*; the resolver controls *what*.** Two separate knobs.
- ⚠️ **Only** `name` **uses the touched/dirty gate.** `price` and `description` still use a plain `{errors.x && ...}`. Works (with `onTouched`), but inconsistent — I should pick one style for all three.
- ⚠️ **The** `id` **in the create body is still the entry-05 white lie** (server ignores it). Unchanged by this work.
- ⚠️ **Leftover:** view mode still says "Error has occured" (typo) and ignores `error.message`.

What's next

- [ ] Use the same touched/dirty error style on all three fields (or just `errors.x`)
- [ ] Reuse `productSchema` to validate the API response (Zod's real superpower)
- [ ] Disable Save until the form `isValid` (needs the `isValid` flag)
- [ ] Fix the "Error has occured" typo / show the real error message

CHAPTER 12

Centralizing Logout (and a CORS Tweak)

2026-07-16

What I set out to do

Two small but satisfying cleanups this session:

1. Add a **Logout button** and make logout live in **one place** — the auth store — instead of being half-duplicated across the app.
2. Loosen the mock server's **CORS** rule so any origin can call it.

Part 1 — A logout button in the authenticated layout

`AuthenticatedLayout` wraps every protected page (via `<Outlet />`, entry 01), so a button here shows on all of them at once:

```
function AuthenticatedLayout() {
  const logout = useAuthStore((s) => s.logout);

  return <>
    Welcome
    <button onClick={logout}>Logout</button>
    <Outlet />
  </>
}
```

`useAuthStore((s) => s.logout)` grabs the `logout` action with a selector — this is a component, so the hook is fine (the rule from entry 10). Notice `onClick={logout}` has **no wrapper** and no `navigate` call — because logout now handles its own redirect. Which is the interesting part...

Part 2 — Moving navigation *into* the store

Originally my logout button did two steps — `logout()` then `navigate({ to: '/login' })` — and my 403 interceptor did the *same* two steps separately. Two copies of "what logout means" is asking for them to drift apart. So I moved the redirect **into the store action itself**:

```
// auth.store.ts
logout: () => {
  set({ token: null, user: null });
  window.location.href = '/login';   // full reload clears all state + query cache
},
```

Now `logout()` is the *one* definition of logging out: clear the auth state, then go to `/login`. Every caller just calls `logout()`.

Why `window.location.href` and not `navigate`?

The store isn't a React component, so it can't use the `useNavigate` hook. Two ways out:

Option	Trade-off
`window.location.href = '/login'`	✅ no imports; **full page reload wipes ALL state + the TanStack Query cache**
`router.navigate(...)`	client-side, but importing `router` into the store risks a **circular import** (store → lib/router → routeTree → routes → auth → store)

For logout, the full reload is actually a *feature* — when a session ends I *want* everything (in-memory state, cached queries) wiped clean. And it dodges the circular-import trap. So `window.location.href` is the right tool here specifically.

> ⚠️ The trade-off I'm accepting: the store now has a **side effect** (navigation), not just pure state. Some folks keep side effects out of stores. For this app the payoff — one definition of logout, shared by the button and the interceptor — is worth it. Just not a pattern to copy into `setAuth` /login, where I *want* smooth client-side `navigate` (which is why `LoginPage` still uses `useNavigate`).

The payoff: deleting the duplicate

Because `logout()` now redirects, the manual redirect in the 403 interceptor was doing the same job twice, so I removed it:

```
 if (error.response?.status === 403) {
-  useAuthStore.getState().logout()
-  window.location.href = '/login';
+  useAuthStore.getState().logout()
 }
```

(Still `.getState()` here, not the hook — the interceptor isn't a component, entry 10.) Now there's exactly one place that knows how to log out, and both callers — the button and the interceptor — go through it. That's the whole point of centralizing.

Part 3 — Allowing any origin (CORS)

Small mock-server change:

```
 app.use(
   cors({
-    origin: 'http://localhost:5173',
+    origin: true,          // reflect whatever origin sent the request (allow any)
     credentials: true,
   }),
)
```

The instinct is `origin: '*'`, but that **won't work here** — my `apiClient` sends `withCredentials: true`, and browsers **forbid the wildcard `*` when credentials are involved** (the CORS gotcha from entry 04). `origin: true` is the correct move: the `cors` package **reflects back whichever origin made the request**, so it accepts any origin *and* stays compatible with credentials.

⚠️ This means *any* website could make authenticated requests to the server — totally fine for a local mock, but something you'd never do on a real backend (there you list specific allowed origins).

Things that tripped me up / notes to self

- **One definition, many callers.** The real win: "log out" is defined once in the store; the button and the 403 interceptor both delegate to it. No drift.
- **Store side effects are a trade-off.** Navigation in a store isn't "pure," but it's pragmatic and DRY here. `window.location` also means the store now assumes a browser (would need mocking in a test).
- **`origin: '*'` ✗ with credentials; `origin: true` ✓.** Reflecting the origin is how you allow "any" while keeping `withCredentials`.
- This finishes the "add a Logout button" item from entry 10's list. ✅

What's next

- [ ] Show `Welcome, {user.name}` in the layout by selecting `user` from the store
- [ ] A real nav bar in `AuthenticatedLayout` (links + logout), not just a bare button
- [ ] Lock CORS back down to specific origins if this ever becomes more than a mock